

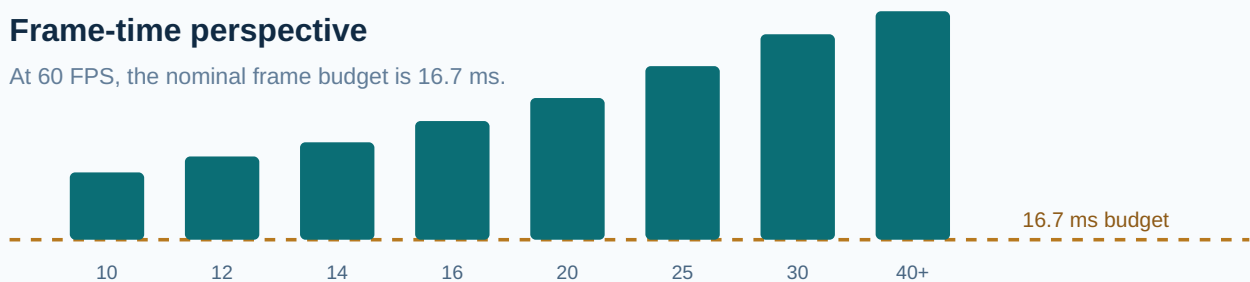
ETS2 1.61 stutter investigation

What the NemoByteCore investigation actually demonstrated, what it did not demonstrate, and what the project is asking SCS Software to address.

Original technical investigation, instrumentation and public repository: NemoByteCore.

Frame-time perspective

At 60 FPS, the nominal frame budget is 16.7 ms.



Illustrative scale. The project reports repeated cold-PSO stalls around 10–40+ ms; the accepted final run lists examples up to 22.351 ms.

1.61.1.1

Investigated build

Euro Truck Simulator 2 · DX12 renderer

10–40+ ms

Measured stalls

Cold graphics-PSO creation/preparation on the draw path

27 / 27

No prior observed async coverage

In the project's accepted final coverage run

wait=0

Diagnostic A/B test

Large stalls disappear, but draws can be skipped / black flashes appear

Purpose. Help journalists, YouTubers, streamers and community channels explain the case accurately without turning a narrow technical result into a universal claim.

This dossier summarises and adapts for public communication the original technical investigation carried out and published by **NemoByteCore**. It is an independent editorial briefing, is not affiliated with SCS Software, and is not presented as an official or endorsed NemoByteCore publication.



Public repository

Executive summary

NemoByteCore, the independent researcher and repository author behind the work summarised in this dossier, instrumented the DX12 rendering path in **Euro Truck Simulator 2 1.61.1.1** and documented a reproducible stutter class involving **graphics Pipeline State Objects (PSOs)** that reach first use “cold.” On the observed path, a cache miss leads to preparation with a synchronous wait, temporarily blocking the latency-critical rendering flow. [1] [2]

The technical work summarised here — reverse engineering, instrumentation, A/B testing, measurements, evidence collection and publication of the repository — belongs to NemoByteCore. This briefing exists to help that work reach a wider audience without losing either its scope or its authorship.

The strongest conclusion supported by the evidence: at least one visible stutter class in the investigated build is engine/renderer-side and requires engine-side work. The investigation does **not** establish that every ETS2 hitch, DX11 problem or ATS stutter has the same root cause.

The final coverage probe compared slow PSOs that later stalled synchronously with an asynchronous preparation path already present in the executable. The accepted result was **27 slow PSOs, with none observed through that async path beforehand.** [1][2]

A controlled test also changed the relevant request from synchronous to asynchronous. The large synchronous stalls disappeared, but missing draws and black flashes occurred while PSOs were still pending. This makes the test **diagnostic evidence, not a user fix**: the expensive work must start early enough for the PSO to be ready before first use. [1][2]

1. What the finding means in plain language

In DX12, a **Pipeline State Object** packages much of the graphics configuration required for a draw: shader programs plus several fixed pipeline states. If the required PSO has not been created/prepared by the time a frame needs it, doing that work on demand can be expensive.

The investigation localized a path where cold PSO creation/preparation can occur with a synchronous wait on the draw path. While that work completes, normal frame production is delayed. The project reports repeated stalls in roughly the **10–40+ ms** range. [1]

Why this is visible even when the FPS counter looks fine. At 60 FPS, the nominal frame budget is 16.67 ms. An unexpected 20, 30 or 40 ms task on a critical rendering path can destroy frame-time consistency even when average FPS remains respectable.

Documented technical paths

```
FUN_1402A06D0
-> FUN_1402AAA60
    -> FUN_14029DD00(..., wait=1)    # cold first use, synchronous wait
```

The same build also contains a path that reaches preparation with `wait=0`:

```
FUN_14029E0F0
-> FUN_14029DD00(..., wait=0)        # asynchronous preparation
```

The `FUN_...` names are reverse-engineering/decompiler labels, not SCS source-level function names, and they may change between builds.

The coverage result at the center of the case

```
async_total = 0
slow_total  = 27
slow_seen   = 0
slow_unseen = 27
```

In that accepted final run, **27 out of 27** measured slow PSOs reached synchronous first use without prior observed coverage from the instrumented async path. [1][2]

2. How NemoByteCore reached that conclusion

- 1 Frame-time correlation.** Spikes were instrumented and followed from broad render-graph timing toward a specific DX12 path.
- 2 Localization.** The dominant stall class was narrowed to a call chain ending in PSO creation/preparation with a synchronous wait.
- 3 Controlled A/B.** Switching the relevant request to `wait=0` removed the large stalls. Incomplete rendering showed that “make it async at the last moment” is not a viable fix.
- 4 Static prewarm rejected.** Capturing slow PSO keys and warming them manually did not generalize: later drives encountered different keys and synchronous warm-up merely moved some stalls elsewhere.
- 5 Async coverage probe.** The existing async preparation path was instrumented and compared against PSOs that later stalled synchronously: 27/27 had no prior observed coverage.

Reasonable interpretation: the engine already contains machinery for asynchronous PSO preparation, but the investigated flow did not get those particular PSOs ready before first use. Reverse engineering alone cannot fully establish whether the internal reason is a regression, incomplete coverage, architectural limitation or something else. [1]

3. What is proven — and what is not

Supported by the evidence

- The investigated target is ETS2 1.61.1.1 running DX12.
- The stall class was reproduced in both vanilla and modded runs; mods were not required.
- Cold-PSO stalls were repeatedly measured at roughly 10–40+ ms.
- The accepted final coverage run recorded 27/27 slow PSOs with no prior observed async coverage.
- Forcing `wait=0` removed the large synchronous stalls but allowed missing rendering while preparation was pending.
- This stutter class therefore requires engine/renderer-side work.

Not supported by the evidence

- That the project has explained *all* ETS2 stutter.
- That the identical mechanism is proven on DX11 or ATS.
- That “27/27” means every PSO in the game.
- That changing `wait=1` to `wait=0` is a usable player patch.
- That SCS is “doing nothing to the engine.” SCS has publicly acknowledged major technical debt and says it is modernising the architecture. [8][9]
- That the technical evidence proves a malicious or commercial motive behind the forum handling.

Recommended headline: “Independent investigation demonstrates a reproducible engine-side DX12 stutter mechanism in ETS2 1.61.1.1.”

Avoid: “We finally know why ETS2 always stutters.”

4. What happened on the official forum

On 1 October 2026, **NemoByteCore** posted the finding on the official SCS forum (under the name NemoByte). The account **MichaelKey**, identified there as **SCS Software**, replied that DX12 remained experimental/unfinished and that DX11 was the supported renderer. When NemoByteCore explained that DX11 was the path he considered practically unusable on his system while the reported finding concerned DX12 stutter, the discussion shifted toward requesting a `game.log` to diagnose the DX11 behaviour. [6]

The thread was later locked and marked **[NOT A BUG]**; the repository also preserves screenshots and an archived snapshot of the exchange. [3][7][10]

The defensible criticism is not “SCS never replied.” There was a response about renderer support and DX11. The more precise point is that the public exchange did **not resolve the narrow 27/27 question**: why did the measured slow PSOs reach first use cold while the same build contained an asynchronous preparation path?

A `game.log.txt` is useful for hardware, configuration, mods, script errors and general support diagnostics. It is not a substitute for profiler timings, PSO creation measurements or targeted coverage instrumentation. The project’s request document therefore asks that instrumented evidence be addressed on its own merits rather than being reduced exclusively to per-machine troubleshooting. [4]

5. The wider context: Prism3D and technical debt

This case does not exist in a vacuum. In SCS's own "Road to Consoles" material, the studio has discussed a codebase built over decades and substantial accumulated technical debt. Petr Šebor publicly described a point where that debt had become large enough to present a choice between rewriting everything or moving forward through incremental changes, and SCS says it is actively modernising the architecture for new platforms. [8][9]

That context cuts both ways. It would be inaccurate to claim that SCS denies all engine debt; the studio has acknowledged it and says work is underway. But that acknowledgement also makes it reasonable to ask for clearer plans around frame-time consistency, render-thread stalls, pipeline preparation/caching, DX11/DX12 direction and regression testing.

The project's community tracker collects public reports across versions and hardware, but explicitly warns that those reports are **not automatic confirmations of the same 1.61.1.1 DX12 mechanism**. Their value is to document recurring stutter complaints and version-to-version smoothness differences, not to inflate the technical claim. [5]

6. What the project is asking SCS to do

1. **Investigate and address the documented cold-first-use PSO path** in ETS2 1.61.1.1 DX12, or show with evidence why the technical interpretation is wrong.
2. **Keep avoidable PSO creation waits off latency-critical rendering paths** by improving preparation, discovery, warm-up and/or persistence where the architecture permits.
3. **Treat frame time as a first-class performance metric**, not just average FPS.
4. **Publish a credible technical roadmap** covering DX11/DX12, frame-time consistency, shader/PSO caching, regression testing and Prism3D modernisation.
5. **Allocate sufficient priority and resources to the technical foundation** that every map, truck, visual feature and DLC depends on. [4]

A serious response can also be a disagreement. If SCS can demonstrate that the async-coverage conclusion is wrong, that would be a valid technical response. The core request is evidence answered with evidence.

7. Creator guide: how to cover this without losing credibility

Do	Avoid
Credit NemoByteCore visibly as the author of the technical investigation and link the original repository.	Presenting the findings as your own, omitting authorship, or linking only to third-party summaries.
Explain the symptom as frame-time spikes that can coexist with acceptable average FPS.	Reducing the issue to “low FPS.”
Use “PSO creation/preparation” and briefly define a PSO.	Calling the entire finding simply “shader compilation” as if the terms were identical.
Repeat the scope: ETS2 1.61.1.1, DX12, one specific stall class.	Generalising the exact mechanism to DX11, ATS or every version.
Present 27/27 as “in the accepted final coverage run.”	Presenting 27/27 as a game-wide population statistic.
Separate measured facts from the manifesto’s strongest opinions.	Assigning motives to SCS that the technical evidence cannot prove.
Acknowledge SCS’s stated engine-modernisation work, then ask for concrete frame-time plans.	Saying SCS “does nothing to the engine.”

Suggested titles

- NemoByteCore traced an ETS2 stutter into the renderer — here is what the data shows
- ETS2 1.61 and the “27 out of 27” finding: a technical breakdown
- Why can ETS2 hitch at high FPS? One measured engine-side case

Short thumbnail text

27/27 NOT PREPARED

10–40+ ms

ENGINE STALL?

YouTube script · approximately 8–12 minutes

Recommended tone: curious, firm and technical. Avoid claims that go beyond what the public evidence establishes.

1. Hook: the FPS counter does not tell the whole story

0:00–0:45

Visual: gameplay apparently locked at 60 FPS; frame-time graph spikes; cut to the GitHub repository.

“If you play Euro Truck Simulator 2, you may know this feeling: the counter says 60 FPS, the road looks smooth, and then the game hitches for a fraction of a second. It is short, but it breaks the rhythm completely. For years, the usual advice has been to check drivers, mods, V-Sync, FPS caps or configuration files. **NemoByteCore** took a very different route: instrument the ETS2 1.61 renderer and follow one class of hitch into the engine. The result does not explain every stutter in the game, but it does demonstrate one specific, reproducible and measurable cause.”

2. What was actually investigated

0:45–1:35

Visual: card: ETS2 1.61.1.1 · DX12 · vanilla + modded · revision/hash in small text.

“The investigation is by NemoByteCore and focuses on a very specific target: Euro Truck Simulator 2 version 1.61.1.1 running DX12. That scope matters. The mechanism we are about to discuss has not been proven as a universal explanation for DX11, American Truck Simulator, or every hitch somebody has ever experienced. This is one documented stall class in one investigated build. The behaviour was reproduced in both vanilla and modded runs, which means mods were not required to trigger the underlying DX12 stall.”

3. What a PSO is — and where the stall happens

1:35–3:05

Visual: Frame → needs PSO → PSO is cold → synchronous creation → stall.

“At the center of the case is something called a Pipeline State Object, or PSO. In DX12, a PSO packages shader programs and several pieces of graphics state the GPU needs for a draw. The problem appears when a required PSO reaches first use cold — it has not been created or prepared yet. The investigation localized a path where a cache miss starts that work and then waits synchronously for it to finish. In other words, the draw path stops and waits. The project repeatedly measured stalls in roughly the 10 to 40-plus millisecond range. For perspective, a 60 FPS frame has a nominal budget of about 16.7 milliseconds. Drop an unexpected 20, 30 or 40 millisecond task into a latency-critical path and average FPS can still look respectable while frame pacing visibly breaks.”

4. The A/B test: remove the wait, change the symptom

3:05–4:15

Visual: split screen: “wait=1” vs “wait=0”; black-flash warning on the async side.

“One of the strongest diagnostic experiments changed the relevant request from a synchronous wait — wait equals one — to an asynchronous request, wait equals zero. The large synchronous stalls disappeared. That strongly supports the localization. But it was not a fix. If the PSO was still being prepared, the draw could be skipped, producing missing rendering and black flashes. So there is no magic config tweak here. The expensive work has to start earlier, so that the PSO is ready before the first draw that needs it.”

5. The 27 out of 27 result

4:15–5:35

Visual: large “27/27” counter and the `async_total` / `slow_total` / `slow_unseen` block.

“This is the result that makes the case especially interesting. The same build already contains an asynchronous PSO preparation path. NemoByteCore instrumented that path and compared it against PSOs that later produced synchronous stalls. In the project’s accepted final run, there were 27 slow PSOs. None of those 27 had been observed through the async preparation path before synchronous first use. The result was slow total 27, slow unseen 27. That does not mean every PSO in ETS2 behaves this way. It means that, in this particular coverage run, every measured slow PSO that later blocked the draw path arrived without prior observed coverage from the instrumented async path.”

6. What this proves — and what it does not

5:35–6:35

Visual: two columns: “Proven” / “Not proven”.

“So can we say we finally know why ETS2 stutters? No. That would overstate the evidence. What we can say is that one reproducible, engine-side DX12 stutter mechanism was demonstrated in ETS2 1.61.1.1: cold PSOs can reach the draw path and trigger a synchronous wait. And in the final coverage run, the measured slow PSOs had not been observed through the existing async preparation path beforehand. What reverse engineering alone cannot fully tell us is why that coverage is missing — whether this is a regression, incomplete wiring, an experimental-renderer limitation or something deeper in the architecture.”

7. The forum response and why the story escalated

6:35–8:00

Visual: official forum screenshots: DX12 experimental, game.log request, [NOT A BUG].

“On October first, the finding was posted on the official SCS forum. MichaelKey, identified on the forum as SCS Software, replied that DX12 is still experimental and unfinished and that DX11 is the supported renderer. NemoByteCore then explained that DX11 was the path he considered effectively unusable on his system, while the reported technical finding concerned DX12. The conversation shifted toward diagnosing DX11 and requesting a game.log. The thread was later locked and marked NOT A BUG, and the repository preserves screenshots and an archived copy. Precision matters here: SCS did respond about renderer support and DX11. What remained unanswered in the public exchange was the narrow technical question — why did those 27 slow PSOs reach first use cold if the build already contains an asynchronous preparation path? A game.log can be useful for many support problems, but it is not a substitute for the timing and coverage instrumentation produced here.”

8. The context creators should also acknowledge

8:00–9:10

Visual: SCS Road to Consoles Dev Talk #2; on-screen text: “technical debt” / “modernising architecture”.

“It would also be misleading to tell this story as if SCS denies that Prism3D needs major work. In its own Road to Consoles material, the studio has discussed a decades-old codebase, substantial technical debt, and ongoing architectural modernisation for new platforms. That matters. This does not have to be framed as ‘SCS does nothing.’ The more useful question is: while that modernisation continues, how will measurable frame-time problems in current builds be treated? What is the plan for PSO and shader preparation, render-thread stalls, regressions between versions and the DX11-to-DX12 transition?”

9. What the project is actually asking for

9:10–10:30

Visual: five requests on screen; repository URL / QR code.

“The reasonable request is not for a player to dictate Prism3D’s architecture. It is not even a demand that SCS immediately support an API it still calls experimental. The request is that technical evidence receives a technical response. If the PSO path has been interpreted incorrectly, show why. If async coverage is missing, fix it. If the limitation is architectural, explain it within a credible modernisation roadmap. And treat stable frame times as a core quality target rather than an endless collection of local tweaks every player is expected to solve alone.”

10. Closing

10:30–11:15

Visual: repository evidence folders; closing text: “Review. Reproduce. Respond.”

“The material is public: the exact build, executable hash, measurements, successful and failed experiments, the 27 out of 27 coverage result and the forum exchange. That means this does not need to become a war of opinions. It can be reviewed, reproduced and argued on technical grounds. If you are a creator, link the original investigation and keep its scope intact. If you are a player, do not claim this explains every stutter you have ever seen. And if this reaches SCS, the core question is simple: what explains those PSOs arriving cold at first use, and what will keep the renderer from stopping to prepare them while the player is driving?”

Technical reference appendix

Item	Value
Game / build	Euro Truck Simulator 2 1.61.1.1
Revision	6949e633e77902f7e023819d3131cc6ccce3707f
Executable SHA-256	EB17944139BE4DE3D70D0CD57CDAA7C52C9E326ECF2D0DD3EA2F806545C74A53
Renderer	DX12
Documented synchronous path	FUN_1402A06D0 -> FUN_1402AAA60 -> FUN_14029DD00(..., wait=1)
Documented async path	FUN_14029E0F0 -> FUN_14029DD00(..., wait=0)
Final coverage result	async_total=0; slow_total=27; slow_seen=0; slow_unseen=27
Representative stalls in the final run	9.958, 10.331, 11.247, 11.617, 11.990, 12.821, 13.303, 14.674, 16.416, 19.936 and 22.351 ms
Scope	The exact mechanism is proven for the investigated ETS2 1.61.1.1 DX12 build. Do not automatically extrapolate it to DX11, ATS or other versions.

This briefing summarises and contextualises public material. The original technical conclusions and evidence should always be checked against the source repository.

Credit and verifiable sources

Original investigation credit: NemoByteCore. The reverse engineering, instrumentation, experiments, measurements, evidence archive and public repository on which this dossier is based are the work of NemoByteCore. Anyone covering this case is encouraged to credit the author by name and link directly to the original repository.

Sources accessed 4 October 2026.

[1] Main investigation / README

<https://github.com/NemoByteCore/ets2-1.61-stutter-investigation>

[2] Accepted ETS2 1.61 runtime evidence archive

<https://github.com/NemoByteCore/ets2-1.61-stutter-investigation/blob/main/evidence/runtime/1.61/README.md>

[3] MANIFESTO.md

<https://github.com/NemoByteCore/ets2-1.61-stutter-investigation/blob/main/MANIFESTO.md>

[4] WHAT_WE_WANT_FROM_SCS.md

https://github.com/NemoByteCore/ets2-1.61-stutter-investigation/blob/main/WHAT_WE_WANT_FROM_SCS.md

[5] Community stutter tracker

https://github.com/NemoByteCore/ets2-1.61-stutter-investigation/blob/main/COMMUNITY_STUTTER_TRACKER.md

[6] Official SCS forum thread

<https://forum.scssoft.com/viewtopic.php?t=354183>

[7] Wayback snapshot of the forum thread

<https://web.archive.org/web/20261002223936/https://forum.scssoft.com/viewtopic.php?t=354183>

[8] SCS Software - Road to Consoles Dev Talk #2

<https://blog.scssoft.com/2026/04/scs-software-road-to-consoles-dev-talk-2.html>

[9] Traxion report/transcript of SCS engine comments

<https://traxion.gg/why-porting-american-truck-simulator-and-euro-truck-simulator-2-to-console-is-a-superhuman-task/>

[10] Repository forum evidence archive

<https://github.com/NemoByteCore/ets2-1.61-stutter-investigation/tree/main/evidence/forum>